Actor Class - Most Important

```
/// <summary>
/// Base class for any participant in combat (Player or Enemy).
/// Holds shared stats, an inventory, learned skills, and the core combat methods that all actors use.
/// </summary>
24 references
public abstract class Actor
{
    // --- Identity ---
    12 references
    public string Name { get; protected set; }

    // --- Core stats ---
    5 references
    public int MaxHp { get; protected set; }
    13 references
    public int CurrentHp { get; protected set; }
    3 references
    public int MaxMp { get; protected set; }
    6 references
    public int CurrentMp { get; protected set; }

    2 references
    public int Strength { get; protected set; }
    2 references
    public int Magic { get; protected set; }
    2 references
    public int Defense { get; protected set; }
    2 references
    public int MagicDefense { get; protected set; }
    2 references
    public int Speed { get; protected set; }
```

Contains the basic parameters of an “Actor” which is a player or enemy object that is to be used for combat.

This script is important because it contains both parameters as well as functions on what skills Actors can use, as well as what those skills do, based on the Combat and Skill namespaces to refer to them easier.

```
1 reference
public virtual bool UseSkill(LearnedSkill learned, Actor target)
{
    if (!learned.TrySpend()) return false;

    Skill skill = learned.Skill;

    switch (skill.EffectType)
    {
        case SkillEffectType.PhysicalDamage:
            target.TakeDamage(EffectiveStrength + skill.Power, isMagic: false);
            break;
        case SkillEffectType.MagicDamage:
            target.TakeDamage(EffectiveMagic + skill.Power, isMagic: true);
            break;
        case SkillEffectType.TrueDamage:
            // Ignores defense entirely; the raw hit lands in full.
            target.TakeRawDamage(EffectiveStrength + skill.Power);
            break;
        case SkillEffectType.Heal:
            Heal(EffectiveMagic + skill.Power);
            break;
        case SkillEffectType.Event:
            // Event skills do nothing on their own; the OnUse callback handles it.
            break;
    }

    skill.OnUse?.Invoke(this, target);
    return true;
}
```

Library Scripts

```
namespace RulerOfTheTomb.Skills
{
    /// <summary>
    /// Static catalog of skill definitions used throughout the game.
    /// Centralizes skill creation so the same definition is reused everywhere.
    /// </summary>
    8 references
    public static class SkillLibrary
    {
        // --- Standard Skills --- //

        /// <summary>
        /// The basic attack every actor knows. Unlimited PP, scales with Strength. Calculates damage based on Defense.
        /// </summary>
        public static readonly Skill BasicAttack = new Skill(
            name: "Attack",
            description: "A standard physical attack.",
            maxPp: -1,
            effectType: SkillEffectType.PhysicalDamage,
            power: 0
        );
```

```
public static class EnemyLibrary
{
    /// <summary>
    /// Scene 2's enemy. Heavy armor, powerful mace, no legs.
    /// </summary>
    1 reference
    public static Enemy CreateLeglessFellow() => new Enemy(
        name: "Legless Fellow",
        maxHp: 1,
        maxMp: 10,
        strength: 38,
        magic: 1,
        defense: 30,
        magicDefense: 26,
        speed: 5
    );
```

The scripts, SkillLibrary, ItemLibrary and EnemyLibrary contain the information for each of the values that can be generally used for objects that are similar.

Since these are all something Actors refer to, to check what skills they can use, or what items they can have in their inventory, or what enemy they are, skills are all readonly and static, so they can always be referred to. Items and enemies are not, so that they can be instantiated when called on, and they don't have to be reset when used or killed.

In the case of Enemies and Items, they use Lambda expressions so I can instance them over and over without giving them a name, meaning that on GameOver they can be reset, as said earlier. These Lambda expressions were a suggestion and implementation made with AI, since I discovered them while trying to make the enemy resetting more efficient, and wasn't aware of what they were or how they worked until I consulted AI on it.

Inventory System

Not everything about the inventory belongs to the inventory script. The script is just something that Actors refer to while in combat to know what they do and don't have. The scripts in the Items folder keeps track of what those items in the Actor inventories are and what they do. And then, the CombatEncounter script keeps track of when to use them.

```
1 reference
private bool PerformItem()
{
    if (!_player.Inventory.HasPouch)
    {
        SceneHelpers.PrintHint("You have no pouch to draw items from.");
        return false;
    }

    List<Usable> usables = _player.Inventory.GetBagUsables();
    if (usables.Count == 0)
    {
        SceneHelpers.PrintHint("Your pouch has no usable items.");
        return false;
    }

    for (int i = 0; i < usables.Count; i++)
        SceneHelpers.Narrate($"{i + 1}. {usables[i].Name}");
    SceneHelpers.Narrate("0. Back");

    Console.Write("> ");
    if (!int.TryParse(Console.ReadLine(), out int pick) || pick < 0 || pick > usables.Count)
        return false;
    if (pick == 0) return false;

    Usable chosen = usables[pick - 1];
    chosen.Use(_player, _enemy);
    _player.Inventory.RemoveFromBag(chosen);
    return true;
}
```

```
3 references
public class Inventory
{
    // --- Equipment slots (always exist, hold one Equipment item each) ---
    private readonly Dictionary<EquipmentSlot, Equipment> _equipment;

    // --- Key items (always exist, no capacity limit) ---
    private readonly List<KeyItem> _keyItems;

    // --- Bag (only functional when HasPouch is true) ---
    private readonly List<Item> _bag;

    /// <summary>
    /// Whether the owner of this inventory has a Pouch.
    /// When false, the Bag cannot be used and Add() will reject non-equipment, non-key items.
    /// </summary>
    13 references
    public bool HasPouch { get; private set; }
```

```
/// <summary>
/// Constructs an empty inventory. Equipment and key-item slots exist by default,
/// but the Bag remains unusable until a Pouch key-item is added.
/// </summary>
1 reference
public Inventory()
{
    _equipment = new Dictionary<EquipmentSlot, Equipment>
    {
        { EquipmentSlot.LeftHand, null },
        { EquipmentSlot.RightHand, null },
        { EquipmentSlot.Armor, null }
    };
    _keyItems = new List<KeyItem>();
    _bag = new List<Item>();
    HasPouch = false;
}
```

Scenes

Scenes use inheritance by splitting them into Normal, Non-Combat scenes and the Final Scene. Normal scenes have combat and exploration, where, after combat, you're forced into the next scene. Non-combat scenes are purely for exploration, they either loop endlessly until you decide to leave, or they force you to take a couple of choices before being forced into the next scene. The Final Scene is a pure combat scene, but in this case, it's also the scene that shows the ending.

In Scenes there are Events, and Events are what contain the choices players can make, as well as the scripts that provide the results of those choices.

```
/// <summary>
/// A single event within a Scene. Events chain together to form the scene's flow.
/// Each event has its own choices, keyword groups, and active state.
/// </summary>
20 references
public abstract class Event
{
    /// <summary>
    /// A short name for this event, used in Help output to label its choice list.
    /// </summary>
    2 references
    public string Label { get; protected set; }

    /// <summary>
    /// Whether this event is the one currently accepting player input.
    /// Only one event in a scene is active at a time.
    /// </summary>
    8 references
    public bool IsActive { get; set; }

    /// <summary>
    /// The choices currently available within this event. Each choice has its own
    /// keyword groups for the parser and a handler that fires when matched.
    /// </summary>
    20 references
    public List<Choice> Choices { get; protected set; } = new List<Choice>();

    /// <summary>
    /// Constructs an event with the given label. Starts inactive.
    /// </summary>
    /// <param name="label">The display label for this event (used in Help output).</param>
    6 references
    protected Event(string label)
    {
        Label = label;
        IsActive = false;
    }
```

```
namespace RulerOfTheTomb.Scenes
{
    /// <summary>
    /// What happened at the end of a Scene's execution. Drives GameSession's
    /// decision on whether to advance, restart, or display an ending.
    /// </summary>
    9 references
    public enum SceneOutcome
    {
        Continue,   // Move to NextScene
        GameOver,   // Player died outside the final boss
        Ending      // Player reached an ending
    }

    /// <summary>
    /// The result of running a scene, passed back to GameSession for next-step decisions.
    /// </summary>
    10 references
    public class SceneResult
    {
        /// <summary>What kind of outcome occurred.</summary>
        2 references
        public SceneOutcome Outcome { get; }

        /// <summary>The next scene to run, when Outcome is Continue.</summary>
        2 references
        public Scene NextScene { get; }

        /// <summary>The ending earned, when Outcome is Ending.</summary>
        2 references
        public Ending Ending { get; }
```

```
/// <summary>
/// The chain of events that make up this scene. Run in order, though branching
/// is allowed, an event can activate a non-sequential next event.
/// </summary>
10 references
public List<Event> Events { get; protected set; } = new List<Event>();
```